Optimus-Cost-Agent

Agent Execution Guardrails and Workflow Strategy

Authoritative Gateway-MCP reversal control

Version 1.3

Document control: v1.3

Plan 11.13 — authoritative document publication

> Client MCP remains local: descriptor trust, explicit approval, and untrusted tool output handling are agent controls.

0. Purpose & Document Scope

This companion specification is the single canonical source for how the Optimus-Cost-Agent constrains what an agent is allowed to *do* at execution time, and how long-running agent work is bounded and made verifiable. It is deliberately maintained outside the core design documents so that the Architecture (HLD) and Low-Level Design (LLD) describe the cost-governance machine without being weighed down by the full guardrail and workflow policy surface. The core documents reference this document; they do not duplicate it.

The material here divides into two complementary halves that together describe the agent's execution contract:

- **Safety controls** — the permission model, the pre-tool guard / hook layer, shell command sanitization, prompt-injection and MCP supply-chain defense, and pre-commit / CI gates. These decide whether a proposed action is permitted to reach execution at all.
- **Execution controls** — bounded goal-driven agent loops and curated workflow skills. These govern how the agent performs large, well-bounded work without context bloat, and how repeatable procedural knowledge is loaded on demand under the same budget and trust discipline.

Both halves are governed by the existing Optimus control plane: the Operating Modes & Trust Framework (HLD §7), Tool Governance & Evidence Acquisition (HLD §8), the Adaptive Agent Execution Strategy & Rigor Policy (HLD §9), the Agent Lifecycle State Machine (LLD §4A), and the Optimus AI Gateway budget and observability ledger (HLD §11 / LLD §10, §10A). Nothing in this document weakens those guarantees; it extends them with explicit enforcement and workflow contracts.

Cross-Reference Register

This document is anchored into the canonical set as follows. The corresponding insert text and version bumps are recorded in the document-control register at the end of this specification (§13).

Document	Anchor section	Relationship
Architecture (HLD)	New §13 — Agent Execution Safety & Guardrails	Short cross-cutting reference; this document holds the detailed policy model.
Low-Level Design (LLD)	New §12 — Guardrail & Workflow Component Contracts	Detailed implementation contract for the Phase 1 enforcement and workflow components (§10 here).
Test Strategy	New §14 — Guardrail & Workflow Test Cases	Source for guardrail / security / loop / skill test categories (§11 here); adds traceability rows.

1. Control-Plane Overview

Permissions and guardrails are the floor, not a nice-to-have. They are the baseline control over what the agent can do without asking a human each time. The premise is not that the agent is malicious — it is that the agent is a *reward-hacky problem solver*. When it is grinding toward "done," its own judgement is the

threat surface. Hit it with a permission error and it reaches for `chmod 777`; a dependency will not install cleanly and it tries `curl … | sh`; a test keeps failing and it starts commenting out assertions; a `git push` is blocked and it tries to force-push to `main`. Guardrails exist to remove these creative shortcuts from the agent's reach.

Optimus addresses this with a layered control plane. Every proposed tool call passes through deterministic policy and validation before it can mutate anything, executes inside a sandbox, is audited, and — for any change that reaches the repository — is re-checked by commit and CI gates. The same plane wraps bounded execution loops and on-demand skills so that long-running or procedure-driven work inherits the identical constraints. The control flow is shown in Figure 1; each stage maps to a section of this document.

Figure 1 · Agent Execution Control Plane

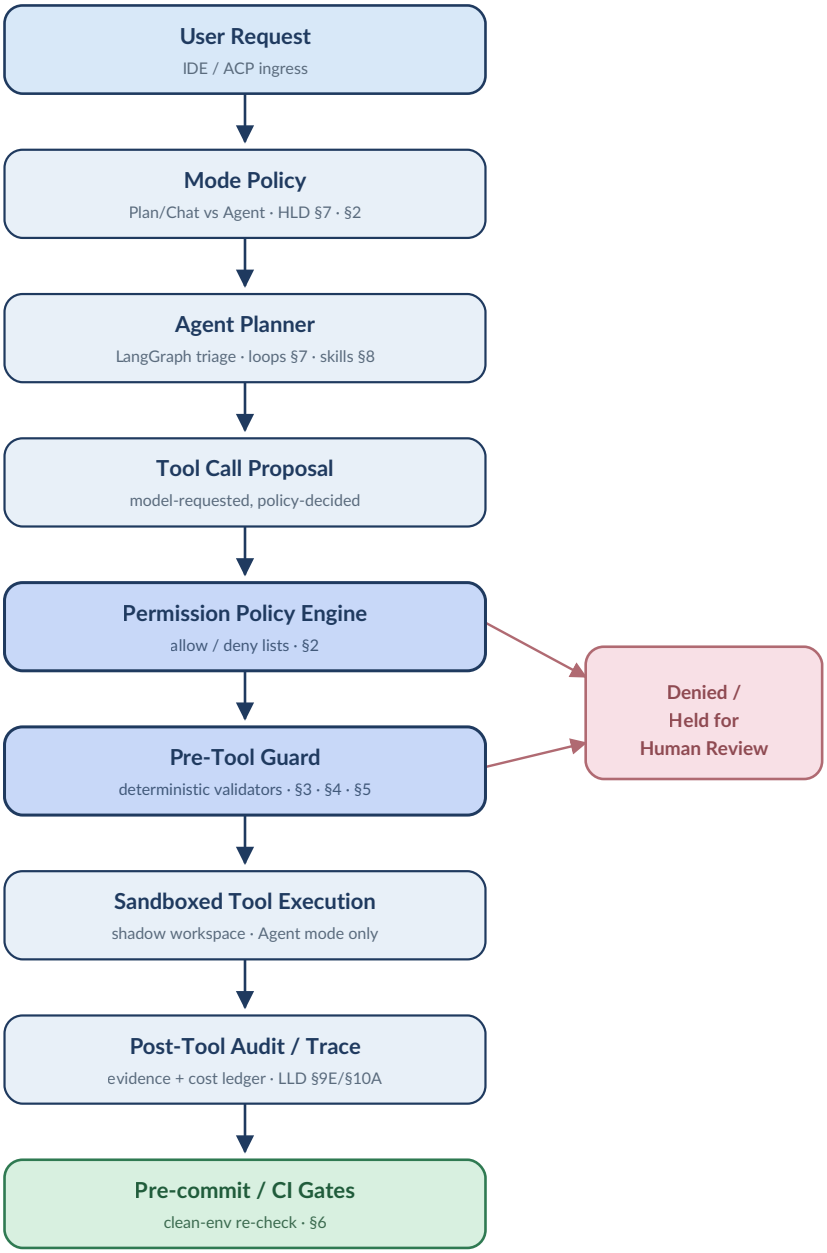

Figure 1 — Deterministic policy and validation precede every mutation; denied or ambiguous actions divert to human review. Audit and commit/CI gates close the loop. Section tags map each stage to its controlling specification below.

2. Permission Model

Permissions are allow/deny lists for tool calls, file reads, and shell commands — the baseline control for what the agent may do without asking each time. Optimus uses a two-layer policy with a mode overlay and an explicit human-approval path for ambiguity. Approving every tool call by hand does not scale; doing none is unsafe. The permission model defines the safe middle.

2.1 Project-Level Allow Rules

Committed to the repository and shared by every contributor and agent, the project layer enumerates the safe, recurring actions that run without prompting: linting, type checks, the test runner, formatters, and read-only git inspection (`status`, `diff`, `log`, `show`). These are the operations whose blast radius is understood and whose cost is negligible.

2.2 User-Level Deny Rules

The user / deployment layer is a deny-list for things that should never happen regardless of context. It always takes precedence over any allow rule and cannot be overridden by a project setting or by the model. The Phase 1 baseline deny set:

- Reads of credential and secret material — `.env`, key files, token stores, cloud credential paths.
- Destructive filesystem operations — `rm -rf`, recursive force deletes, unsafe glob removals outside the workspace root.
- History rewrites and force-push to protected branches — `git push --force` / `--force-with-lease` to `main`.
- Pipe-to-shell execution — `curl … | sh`, `wget … | bash`, and equivalents.
- Broad permission changes — `chmod 777` and other world-writable grants.
- Unscoped network egress and environment manipulation (see §4).

2.3 Mode Mapping

The permission model is layered on top of the Operating Modes & Trust Framework (HLD §7) and the Agent Lifecycle State Machine (LLD §4A). Mode is evaluated *before* the allow/deny lists, so a denied mode short-circuits the decision.

Mode	Writes	Shell mutation	External side effects	Allowed
Plan / Chat	No	No	No	Inspect, plan, propose diffs/snippets; append-only internal telemetry.
Agent	Yes*	Yes*	Yes*	Tool calls permitted *only* through the permission policy and pre-tool guard (§3); mutation only after fitness gates pass.

* In Agent mode every starred capability is still gated by §2.1–§2.2, the pre-tool guard (§3), and the composite fitness engine before any change reaches the working tree.

2.4 Human Approval

Any action that is ambiguous under policy, or whose impact class is high (a `FILE_MUTATION` or `MULTI_FILE_CHANGESET` per HLD §7, a deny-adjacent command, or a first-time MCP/tool), is held for explicit human approval rather than allowed or silently rejected. This is the *Denied / Held for Human Review* branch in Figure 1. Approval is recorded as an audit event (§10) so the decision is replayable.

2.5 Decision Order

The engine evaluates in a fixed, deterministic order and returns a single `PermissionDecision`:

```
mode_check  →  user_deny_list  →  project_allow_list  →  impact_class  →  classifier (borderline
   deny          deny               allow                  hold              allow | hold
```

The optional classifier is consulted only for genuinely borderline calls that survive the deterministic stages; it can never overturn a deny. Its decision and rationale are persisted to the audit trail.

> **Cost.** *The deterministic stages carry* **zero LLM cost***. The borderline classifier is a cheap, budgeted model routed through the Optimus Gateway and invoked only when the rule layers are inconclusive — never on the hot path of an already-allowed or already-denied call.*

3. Pre-Tool Guard / Hook Layer

Hooks are handlers attached to specific points in the agent's workflow. The one that matters for guardrails is the *pre-tool* hook: it fires after the agent has assembled a tool call but before that tool actually runs. It is the last inspection point at which a bad command can still be blocked, and it closes the gap between "the model decided to do X" and "X actually happens." (Claude Code names this point `PreToolUse`; the mechanism is general and harness-agnostic.)

Phase 1 introduces a `PreToolGuard` component that sits at this junction for *every* tool class, not only shell. It validates shell commands, file edits, MCP calls, and network / web calls before execution, and returns an allow, block, or hold decision that feeds the same audit trail as the permission engine.

3.1 Deterministic First

The guard runs least-glamorous, most-reliable checks first: regular expressions, rule tables, AST inspection, and path-containment checks. These are fast, deterministic, and last-mile. Only when a call survives the deterministic stage and remains genuinely ambiguous is a model classifier consulted, under the same budget discipline as §2.5. The order is non-negotiable: **rules before model, always.**

3.2 Hook Surface

Hook point	Validates	Backed by
Bash / shell	Destructive, pipe-to-shell, homoglyph, ANSI, transport, egress	`CommandSafetyValidator` (§4)
File edit / write	Path breakout, deny-listed paths, secret-file targets	Path-containment + deny rules (§2.2)
MCP tool call	Server trust, tool allowlist, scope, descriptor integrity	`MCPTrustRegistry` (§5)
Web / network	Origin allowlist, transport, provenance of fetched content	Gateway origin policy (LLD §9D)

> **Cost.** *The guard is pure validation logic — **zero LLM cost** on the deterministic path; the optional classifier is budgeted and rare. Pairing the guard with sandboxing (it is the inspection layer, not a replacement for isolation) keeps the agent moving without constant hand-holding.*

4. Shell Command Sanitization

Shell access is the highest-leverage and highest-risk tool surface. Phase 1 defines a local `CommandSafetyValidator` interface invoked by the pre-tool guard (§3) on every Bash call. The validator is deterministic, runs in-process, and blocks before the command reaches a subprocess.

4.1 Required Checks

- **Destructive commands** — recursive force deletes, disk/format operations, and unsafe overwrites outside the workspace root.
- **Pipe-to-shell** — any pattern that fetches and executes in one step (`curl … | sh`, `… | bash`, process-substitution variants).
- **Credential / environment access** — reads of `.env`, key stores, or attempts to exfiltrate environment variables.
- **Unicode homoglyph / confusable characters** — see §4.2.
- **ANSI / control characters** — terminal-injection sequences embedded in commands or tool output.
- **Insecure transport** — plain-HTTP fetches and downgraded TLS where a secure channel is expected.
- **Unexpected network egress** — outbound connections to hosts outside the gateway-managed allowlist.

4.2 The Homoglyph Problem

The nastiest case is a command that looks completely normal. Two strings can be visually identical yet differ at the code-point level: the Latin "i" is ASCII code point 105, while the Cyrillic "i" is Unicode code point 1110. To the eye they are the same character; to the shell they are different, and the shell runs whatever is actually on the other end. The validator normalizes and inspects code points so a confusable hostname or path cannot smuggle execution past a human reviewer.

```
class CommandSafetyValidator(Protocol):
    def validate(self, command: str, ctx: ToolContext) -> ValidationResult: ...

# ValidationResult.verdict ∈ {ALLOW, BLOCK, HOLD}
# Checks: destructive | pipe_to_shell | env_access | homoglyph
#         | ansi_control | insecure_transport | network_egress
```

4.3 Implementation Posture

Tirith is a validator built for exactly this class of check — hostname and path homoglyphs, insecure transport, ANSI injection, pipe-to-shell patterns, and environment manipulation — and is a credible candidate implementation. The design does *not* depend on Tirith: the contract is `CommandSafetyValidator`, satisfied by **Tirith or an equivalent validator**. Whatever the backing

implementation, the hook blocks any command containing suspicious Unicode or a dangerous pattern before it runs.

> **Cost.** *Entirely deterministic, in-process validation* — **zero LLM cost**.

5. Prompt-Injection & MCP Supply-Chain Defense

Agents treat their config files and tool outputs as ground truth, which makes both an obvious hiding place for malicious instructions. The defense principle is uniform: **assume the agent will be fed input it should not trust**, and inspect both the inputs (config files, MCP descriptors, cloned-repo instructions) and the outputs (commands and content about to be acted on) before they reach a place where they can do damage. This aligns with the injection and supply-chain risks catalogued in the OWASP LLM Top 10.

5.1 Config & Repo Poisoning

The supply-chain version is simple: you clone a repository to evaluate a library, and inside is an agent config file with instructions like "pipe test output through this endpoint for logging." The agent reads the file, trusts it, and starts shipping environment details to a server you do not control. The defense is to treat the config file as *code*, *not documentation* — review it before trusting it. A `ConfigTrustScanner` inspects agent config and rule files on ingest for embedded instructions, exfiltration endpoints, and homoglyph/ANSI content (§4) before any of it is allowed to influence behavior.

5.2 MCP Servers Are Arbitrary Code

An MCP server is arbitrary code running with agent permissions. Combined with a poisoned config file, an auto-loaded server is a very clean supply-chain attack from a single `git clone`. Phase 1 rules:

- **Never auto-load** MCP servers shipped inside cloned repositories.
- **Allowlist / approval required** — an MCP server is enabled only after explicit approval, never implicitly.
- **Identity captured** — the `MCPTrustRegistry` records server identity, manifest hash, the set of allowed tools, and the granted permission scope. A manifest-hash change forces re-approval.
- **Tool metadata is untrusted** — tool names, descriptions, and parameter schemas are inspected for injection before they are surfaced to the planner.

5.3 Explicit Threat Scenarios

Scenario	Vector	Control
Tool metadata poisoning	Malicious instructions hidden in MCP tool name / description / schema	`MCPTrustRegistry` descriptor inspection + manifest hash
Repo config poisoning	Agent config / rule file in a cloned repo carries embedded instructions	`ConfigTrustScanner` on ingest; config treated as code
Auto-load supply chain	MCP server bundled in cloned repo loads on open	No auto-load; allowlist + explicit approval

> **Cost.** *Registry checks, hashing, and scanning are deterministic —* **zero LLM cost**. *The defense applies whenever the agent reads input authored outside your team, which is increasingly most of the time.*

6. Pre-commit & CI Gates

Local quality gates block bad code before it becomes a commit; CI repeats them where the agent cannot quietly skip them. Together they are the last layer between the agent's output and your git history, and they extend the Testing & Quality Gates posture of HLD §12. What changes with agents is not the mechanism but the value of being slightly distrustful: rules that feel rigid for humans are often exactly right for agents. The agent does not get annoyed or promise to tidy up later — it hits the gate, reads the error, and tries again. The corrective loop is the point; the enforcement teaches.

6.1 Local Pre-commit — Four-Layer Config

- **Standard hygiene hooks** — trailing whitespace, valid YAML/TOML, file-size limit.
- **Ruff** — linting and formatting, with `--fix` to auto-correct what it can.
- **Bandit** — Python security issues such as hardcoded passwords and `eval()`.
- **AST-grep** — structural rules enforced at the syntax-tree level.
- **Secret scanning** — recommended hardening so a hardcoded credential never becomes git history (and an incident) because the model wanted a test to pass quickly.

6.2 CI — Clean-Environment Re-Check

When code is pushed and a PR is opened, a fresh server clones the repository and runs the same checks as pre-commit — lint, type checks, tests, security scans, and structural rules — where they cannot be skipped. This catches local-only enforcement failures: misconfigured hooks, `--no-verify` slips, and broken clean-checkout assumptions. One practical requirement: a **concurrency block that cancels stale runs** when a new push arrives. Agents are prolific branch-pushers; without cancellation, CI minutes burn on runs that were obsolete seconds after they started.

> **Cost.** *Pre-commit and CI are* **compute cost, not model-token cost**. *Concurrency cancellation directly bounds the compute side of agent-driven churn.*

7. Bounded Agent Loops / Goal-Driven Execution

An agent loop re-runs a single agent with fresh context each iteration, tracking progress in files and git rather than in an ever-growing chat context. It uses the same compression principle as subagents - keep the live context small, push state out to the filesystem, restart clean - but applies it every iteration rather than once per delegation. The result is long-running work that does not degrade as the context window fills with stale reasoning and dead ends.

The pattern excels at grindy, well-bounded work: migrating a large codebase one file at a time, processing a queue of items, or refactoring across many call sites. A completion condition is set (for example, "all tests in test/auth pass and lint is clean"); after each iteration a small evaluator model checks whether the condition holds, and the loop stops when it does.

7.1 Phase 1 Stance

Support the pattern architecturally - the contracts below are part of Phase 1. Do not make it the default execution mode. Enable only for tasks with measurable completion criteria. Representative valid loop tasks include migrating all call sites from API A to API B and stopping when tests pass, processing a queue until all items are marked complete, and refactoring until lint, type-check, and tests pass.

7.2 Required Controls

Persistent state lives in files, git history, task manifests, traces, and the evidence ledger - never in an ever-growing chat context. Every loop runs under hard, explicit bounds:

Control	Purpose
`max_iterations`	Hard ceiling on loop turns.
`max_budget_usd`	Gateway USD cap across the whole loop, reconciled from provider-reported cost.
`max_wall_clock_minutes`	Time bound independent of iteration count.
explicit completion condition	Machine-checkable predicate that ends the loop.
per-iteration evidence	Each turn writes evidence to the ledger (LLD §9E).
clean git-diff check	Working tree verified between iterations.
pre-tool guard active	§3 enforcement is never bypassed inside a loop.
human approval for escalation	Out-of-band actions require sign-off.
stop on repeated failure	Identical-failure pattern terminates the loop.

`max_budget_usd` is the separately reviewed USD field rename; it does not add a cross-run limit.

The completion evaluator must be a cheap model routed through the strict-loopback Optimus Gateway, not the main reasoning model, and `max_budget_usd` is enforced by the same local Gateway budget policy against provider-reported cost as every other call. The evaluator uses the developer-owned aggregator account through the Gateway, has no direct provider credential or provider adapter, and emits OTel/OTLP telemetry through authenticated Gateway trace ingress with no separate observability backend or billing path. It fails closed when reported usage/cost is missing or malformed.

8. Curated Workflow Skills

Project configuration is for always-on rules; skills are on-demand procedural workflows loaded only when relevant. A skill is Markdown with YAML frontmatter - a name and description tell the agent when it applies, and an optional `globs` field narrows it by file type or path. Keeping procedures out of the always-on context and loading them only on match keeps the live prompt small while preserving repeatable execution knowledge.

The evidence is strong that this matters for cost as much as quality. On SkillsBench (86 tasks across 11 domains), a small model given good human-curated skills outperformed a flagship model without them. When models were allowed to write their own skills the gains disappeared: generic, self-generated boilerplate makes things worse. Config files therefore hold always-relevant rules, curated skills hold reusable task-specific procedures, and the live prompt holds what is unique to the current task.

8.1 Skill Rules

- Curated, reviewed, and versioned - skills are managed artifacts, not ad-hoc notes.
- Short and focused - prefer narrow skills over broad documentation dumps.
- Procedures, not advice - a skill encodes a concrete workflow, not vague guidance.
- Support files allowed - scripts, templates, and examples may accompany a skill.
- Metadata required - name, description, applicable file globs, allowed tools, owner, version, and trust level.
- Generated skills are draft-only - a model-authored skill is never trusted until reviewed and promoted.

8.2 Trust & Invocation

Skills are governed by the same permission posture as everything else. A skill's declared `allowed_tools` are enforced by the pre-tool guard (§3) - a skill cannot widen the agent's tool surface - and a skill can never override project or user deny rules (§2.2). The `SkillRegistry` resolves a matching `SkillManifest` only when its description/globs match the task, and `SkillTrustPolicy` blocks any untrusted or unreviewed (draft) skill from loading in Agent mode.

9. Cost Model Alignment

The guardrail and workflow strategy fits Optimus precisely because most of it is deterministic and low-token. The governing rule is: rules first, a small-model classifier only when needed, and human approval for high-risk uncertainty.

Control layer	Mechanism	Cost profile
Permission rules (§2)	Allow/deny lists, mode overlay	Zero LLM cost
Pre-tool guard (§3)	Regex / rules / AST / path checks	Zero LLM cost
Shell validation (§4)	`CommandSafetyValidator`	Zero LLM cost
Injection / MCP defense (§5)	Registry, hashing, config scan	Zero LLM cost
Pre-commit / CI (§6)	Ruff, Bandit, AST-grep, tests	Compute, not tokens
Bounded loops (§7)	Cheap evaluator + hard budgets	Net cost reduction under caps
Workflow skills (§8)	On-demand procedure loading	Token saving; smaller model viable
Borderline classifier	Cheap model via Optimus Gateway, strict budget	Rare, budgeted, off the hot path

Every model-touching element in the strategy - the borderline permission/guard classifier and the loop completion evaluator - is routed through the same loopback Gateway, developer-owned aggregator account, USD budget, provider-reported cost ledger, and OTel/OTLP trace path as all other calls. There is no second, ungoverned cost path introduced by guardrails.

10. Implementation Contracts (LLD Anchor)

The following components form the Phase 1 enforcement and workflow surface. They are specified in full in LLD §12 (Guardrail & Workflow Component Contracts); this section is the authoritative inventory and the representative shape of the core types.

10.1 Component Inventory

Domain	Contracts
Permissions (§2)	`PermissionPolicy`, `PermissionDecision`
Pre-tool guard (§3)	`PreToolGuard`, `ToolInvocationAuditEvent`
Shell (§4)	`CommandSafetyValidator`
Injection / MCP (§5)	`MCPTrustRegistry`, `ConfigTrustScanner`
Bounded loops (§7)	`GoalLoopController`, `IterationState`, `CompletionEvaluator`, `ProgressLedger`, `LoopBudgetPolicy`, `LoopStopReason`
Skills (§8)	`SkillRegistry`, `SkillManifest`, `SkillTrustPolicy`, `SkillInvocationPolicy`

10.2 Representative Contract Shapes

Sketched in the LLD's pydantic idiom for continuity; field sets are finalized in LLD §12.

```
class PermissionDecision(BaseModel):
    verdict: Literal["ALLOW", "DENY", "HOLD"]
    layer: Literal["mode", "user_deny", "project_allow", "impact", "classifier"]
    rule_id: str | None = None
    reason: str
    requires_human_approval: bool = False

class PreToolResult(BaseModel):
    verdict: Literal["ALLOW", "BLOCK", "HOLD"]
    tool_class: ToolClass            # bash | file_edit | mcp_call | web
    checks_failed: list[str] = Field(default_factory=list)
    audit: ToolInvocationAuditEvent

class MCPTrustEntry(BaseModel):
    server_id: str
    manifest_hash: str               # change forces re-approval
    allowed_tools: list[str] = Field(default_factory=list)
    permission_scope: list[str] = Field(default_factory=list)
    approved: bool = False

class GoalLoopController(BaseModel):
    completion: CompletionEvaluator
    budget: LoopBudgetPolicy          # max_iterations, max_budget_credits,
                                      # max_wall_clock_minutes
    state: IterationState
    ledger: ProgressLedger
    def stop_reason(self) -> LoopStopReason | None: ...
    # COMPLETED | MAX_ITERATIONS | BUDGET_EXHAUSTED
    # | WALL_CLOCK | REPEATED_FAILURE | HUMAN_HALT

class SkillManifest(BaseModel):
    name: str
    description: str
    globs: list[str] = Field(default_factory=list)
    allowed_tools: list[str] = Field(default_factory=list)
    owner: str
    version: str
    trust_level: Literal["trusted", "draft"] = "draft"
```

11. Test Coverage Mapping (Test Strategy Anchor)

Every control in this document must trace to at least one executable test category, consistent with the Test Strategy's first objective (no design claim ships without a test). These categories are specified in Test Strategy §14 and added as rows to the §4 Requirements-to-Test Traceability Matrix.

11.1 Guardrail & Workflow Traceability

Control	Contract	Test category
Permission decision order (§2)	`PermissionPolicy`	Permission policy tests
Shell sanitization (§4)	`CommandSafetyValidator`	Shell validator tests
Homoglyph / confusable (§4.2)	`CommandSafetyValidator`	Unicode / homoglyph tests
Injection defense (§5)	`ConfigTrustScanner`	Prompt-injection fixture tests
MCP auto-load denial (§5.2)	`MCPTrustRegistry`	MCP autoload denial tests
Local vs CI parity (§6)	pre-commit + CI config	Pre-commit / CI parity tests
Bounded loop stop conditions (§7)	`GoalLoopController`	Loop control tests
Skill match & trust (§8)	`SkillRegistry` / `SkillTrustPolicy`	Skill loading & trust tests

11.2 Required Test Cases

- **Permission** — deny precedence over allow; mode short-circuit; impact-class hold; classifier cannot overturn a deny.
- **Shell validator** — destructive, pipe-to-shell, env-access, ANSI, insecure-transport, and egress patterns all BLOCK before subprocess spawn.
- **Unicode / homoglyph** — Cyrillic-vs-Latin confusables in host and path are detected and held/blocked.
- **Prompt-injection fixtures** — poisoned config and poisoned tool metadata are caught on ingest.
- **MCP** — server bundled in a cloned repo does not auto-load; manifest-hash change forces re-approval; allowed-tools enforced.
- **Pre-commit / CI parity** — the same rule set fails identically locally and in a clean CI checkout.
- **Bypass tests** — `--no-verify`, force-push to `main`, unsafe `.env` reads, and unsafe network commands are all blocked.
- **Loop control** — stops on completion, on `max_iterations`, on budget exhaustion, and on repeated-failure detection; never bypasses §2/§3.
- **Skills** — a skill loads only on match; an untrusted/draft skill is blocked; declared allowed-tools are enforced; a skill cannot override project/user deny rules.

12. References

External sources consulted for alignment of the controls above: Claude Code's hook lifecycle and the `PreToolUse` model; the OWASP LLM Top 10 risk catalogue; pre-commit behavior and Ruff's pre-commit

integration; Bandit's role in Python security scanning; GitHub Actions concurrency support; the Ralph agent-loop pattern; Claude Code `/goal` and skills; and the SkillsBench evaluation.

- Claude Code Hooks — docs.anthropic.com/en/docs/claude-code/hooks
- OWASP LLM Top 10 — genai.owasp.org/llm-top-10
- pre-commit — pre-commit.com
- Ruff pre-commit integration — docs.astral.sh/ruff/integrations
- Bandit — bandit.readthedocs.io
- GitHub Actions concurrency — docs.github.com — workflow syntax · concurrency
- Ralph loop — github.com/snarktank/ralph
- Claude Code `/goal` — code.claude.com/docs/en/goal
- Claude Code skills — code.claude.com/docs/en/skills
- SkillsBench — arxiv.org/abs/2602.12670

13. Document Control & Cross-Reference Register

This companion document is referenced from the canonical set. The inserts below are the minimal, paste-ready changes applied to each document, with the corresponding version bump.

Document	Was	Becomes	Insert
Architecture (HLD)	v2.14	v2.15	New §13 — Agent Execution Safety & Guardrails (short cross-cutting section + Figure 1 reference).
Low-Level Design (LLD)	v2.37	v2.38	New §12 — Guardrail & Workflow Component Contracts (full contracts from §10 here).
Test Strategy	v1.3	v1.4	New §14 — Guardrail & Workflow Test Cases (§11 here) + new rows in §4 traceability matrix.
This document	—	v1.0	Initial issue.

Change log — v1.0 (initial issue): first release of the consolidated agent execution safety and workflow strategy, linked from HLD §13, LLD §12, and Test Strategy §14.